

Image_Capture.ipynb

```
from io import IncrementalNewlineDecoder
from sentence_transformers import SentenceTransformer
from PIL import Image
import matplotlib.pyplot as plt
import numpy as np
%matplotlib inline
model = SentenceTransformer("clip-ViT-B-32")
```

```
img1 = "/content/5.jpeg"
img1 = Image.open(img1)
plt.figure(figsize = (6,6))
plt.imshow(img1)
plt.axis("off")
plt.title("Image For Similiarity Comparison",fontsize=14)
plt.show()
```

```
text_description = ["Two dogs in the snow",
                    "A Cat on a table",
                    "A Picture of london at night",
                    "A dog sleeping on a couch",
                    "Two Cats fighting in the Garden",
                    "This image features Black Panther, the Marvel
superhero, in a dramatic watercolor splash-art style",
                    "This image is a close-up side profile of a
futuristic humanoid robot with a sleek white outer shell",
                    "The robot has a human-like face with smooth
features and a calm expression",
                    "This image shows a striking black wolf looking
directly at the viewer",
                    "The peacock stands proudly, often associated with
royalty and divine elegance",
                    "Known as the "Palace of Winds," this Jaipur
landmark features a honeycomb façade of pink and red sandstone. ",
                    "A serene marble structure dedicated to Lord
Krishna, adorned with ornate carvings and domes" ,
                    "Its massive walls, watchtowers, and gateways tell
stories of valor and royal legacy, overlooking arid landscapes and
ancient towns.",
                    "It beautifully combines the elegance of a
peacock, the grandeur of the Hawa Mahal",
                    "Each element is placed in harmony, creating a
```

vivid snapshot of India's heritage."

]

```
def display_image_and_compare(image_path, text_description):

    # loading image
    img = Image.open(image_path)

    # Encode Text and Image (Converting both into Numbers)
    text_emb = model.encode(text_description)
    img_emb = model.encode(img)

    # Compute Similarity Score
    similarity_score = np.dot(text_emb, img_emb.T)

    # Find the Best Match
    best_match_index = np.argmax(similarity_score)
    best_match_desc = text_description[best_match_index]

    # Print the Final Output
    print(f"\nBest match for the image is = {best_match_desc} = with
score of = {similarity_score.max()}")

    # Display the Image
    plt.figure(figsize = (6,6))
    plt.imshow(img1)
    plt.axis("off")
    plt.title("Image For Similiarity Comparison", fontsize=14)
    plt.show()

display_image_and_compare("/content/5.jpeg", text_description)

img1 = "/content/3.jpeg"
img1 = Image.open(img1)
plt.figure(figsize = (6,6))
plt.imshow(img1)
plt.axis("off")
plt.title("Image For Similiarity Comparison", fontsize=14)
plt.show()
```

```
display_image_and_compare("/content/3.jpeg",text_description)
```

```
img1 = "/content/6.jpeg"  
img1 = Image.open(img1)  
plt.figure(figsize = (6,6))  
plt.imshow(img1)  
plt.axis("off")  
plt.title("Image For Similiarity Comparison",fontsize=14)  
plt.show()
```

```
display_image_and_compare("/content/6.jpeg",text_description)
```

```
img1 = "/content/7.jpeg"  
img1 = Image.open(img1)  
plt.figure(figsize = (6,6))  
plt.imshow(img1)  
plt.axis("off")  
plt.title("Image For Similiarity Comparison",fontsize=14)  
plt.show()
```

```
display_image_and_compare("/content/7.jpeg",text_description)
```

```
img1 = "/content/8.jpeg"  
img1 = Image.open(img1)  
plt.figure(figsize = (6,6))  
plt.imshow(img1)  
plt.axis("off")  
plt.title("Image For Similiarity Comparison",fontsize=14)  
plt.show()
```

```
display_image_and_compare("/content/8.jpeg",text_description)
```

```
img1 = "/content/9.jpeg"  
img1 = Image.open(img1)  
plt.figure(figsize = (6,6))  
plt.imshow(img1)  
plt.axis("off")  
plt.title("Image For Similiarity Comparison",fontsize=14)  
plt.show()
```

```
display_image_and_compare("/content/9.jpeg",text_description)
```

```
img1 = "/content/10.jpeg"
img1 = Image.open(img1)
plt.figure(figsize = (6,6))
plt.imshow(img1)
plt.axis("off")
plt.title("Image For Similiarity Comparison",fontsize=14)
plt.show()
```

```
display_image_and_compare("/content/10.jpeg",text_description)
```

```
img1 = "/content/11.jpeg"
img1 = Image.open(img1)
plt.figure(figsize = (6,6))
plt.imshow(img1)
plt.axis("off")
plt.title("Image For Similiarity Comparison",fontsize=14)
plt.show()
```

```
display_image_and_compare("/content/11.jpeg",text_description)
```

```
img1 = "/content/13.jpeg"
img1 = Image.open(img1)
plt.figure(figsize = (6,6))
plt.imshow(img1)
plt.axis("off")
plt.title("Image For Similiarity Comparison",fontsize=14)
plt.show()
```

```
display_image_and_compare("/content/13.jpeg",text_description)
```